段冰 Reese

+86 18510840710 | reese_duan@163.com | github.com/imReese | 技术博客

求职方向 AI Infra / 大模型推理系统

专注大模型推理系统开发与生产环境性能优化，覆盖多模型接入、缓存子系统建设与运行时调度；具备 CPU 微架构性能分析和分布式存储控制面开发经验。

工作经历

百度 · AI 计算 / 训推引擎组 2025.06 - 至今

软件工程师（大模型推理引擎）

负责百度智能云千帆 MaaS 的推理引擎开发与性能优化，以及 SGLang 的生产部署与工程化，支撑多模型在线推理服务。

多模型接入与生产稳定性

- 完成 Kimi 系列引擎适配及 PP 部署适配，支撑 GLM、DeepSeek 系列落地与精度验证；修复 PP 部署中的稳定性问题。
- 完善引擎可观测性接口，定位并解决核心稳定性问题。

Attention Store：模型状态管理与缓存复用

- 面向多模型、多并行部署中的状态复用，完成 Attention Store（AS）缓存与统一 radix 树的合并迁移，建成缓存存储子系统，并解决多并行、多模态场景中的稳定性问题。
- 优化 AS 读写路径和前缀复用，提升缓存命中率和 Prefill 吞吐；团队 AS 方案支持 KDA 状态及 PP 下的 L3 存储接入。

推理运行时：传输计算协同与执行优化

- 将主机侧 KV cache 操作、H2D/D2H 传输与 GPU 批次执行组织为异步流程，实现传输与计算重叠；结合 Mooncake TransferEngine 分析与优化 KV 传输路径。
- 承担 Prefill/EPD 与约束解码相关优化；针对定长压测与真实流量的性能差异，分析请求长度和 KDA 后端选择对 Prefill 性能及缓存复用收益的影响。

分层缓存探索：探索 LMCache 与 KV manager 的分层架构，搭建服务端并验证方案可行性。

华为云 · 竖亥实验室 2023.07 - 2024.07

计算机体系结构研究工程师

面向云业务负载开展 CPU 微架构分析与软硬件协同优化。

- **云负载特征分析**：围绕 MySQL、Redis 等云服务，基于 perf 和 Intel Pin 构建低开销指令流分析工具，提取负载特征，为 CPU 微架构方案验证提供依据。
- **微架构方案验证**：结合负载特征与 ChampSim 仿真验证 CPU 微架构优化方案，TLB 命中率提升 30%，IPC 提升 6%；相关方案已申请发明专利。
- **软硬件协同优化**：与海思、openEuler 团队协作验证鲲鹏 CPU 优化方案，单核吞吐量提升 8%。

华为数据存储产品线 · 基础设施开发部 2022.05 - 2023.07

软件开发工程师

- **控制面开发**：负责分布式存储 Cluster Manager，覆盖服务选主、客户端选主与集群生命周期管理。
- **异步化与规模扩展**：通过异步消息队列和模块解耦，将单集群支持节点数从 128 扩展至 256，资源初始化效率提升 40%。
- **可靠性建设**：主导 200+ 接口自动化测试与安全加固，分支覆盖率 90%+；定位并解决 20+ 生产环境高优先级问题。

段冰 Reese

AI Infra / 大模型推理系统

独立项目

独立开展推理运行时、状态复用系统和推理控制面的设计与实现。

SGLang Rust Runtime 2026.05 - 至今

github.com/imReese/sglang-rs Rust / gRPC / CUDA

- **批次调度**：按未缓存 token 数控制 Prefill 批次规模，支持页对齐的前缀复用；P/D 分离模式下，Decode 等待 KV 就绪后执行，异步 Prefill 的缓存页在传输完成后才对外可用。
- **统一物理 KV 页**：CUDA 执行与 Mooncake 注册共用同一 page-major KV pool，通过 RAII 管理注册和设备内存，确保先注销再释放；失败时回收尚未对外提供的缓存页，请求取消时同步取消相关传输。
- **执行后端**：实现 Qwen2/3 共享 dense decoder、F32 CPU 参考与 BF16 CUDA 执行路径，接入 cuBLAS GEMM 与 GQA paged attention。

NexusKV · 跨引擎状态复用 2025.04 - 至今

github.com/imReese/NexusKV Go / Rust / Python

- **复用条件与边界**：设计跨引擎复用层，校验租户、模型版本与数据布局，评估复用范围、传输成本和引擎使用条件，并记录依据；复用判断与存储传输解耦，由引擎负责内存分配并决定是否使用缓存。
- **索引与适配**：用 Rust 实现前缀索引和部分复用计划，以写时复制处理并发更新；通过 Host DRAM 存储及引擎适配接口协调状态传输、使用与释放。

Locus · 状态感知的推理控制面

github.com/imReese/Locus Rust / Axum

- **计算与缓存协同调度**：设计跨实例推理控制面，按模型语义和引擎能力筛选可用实例，再结合租户策略、实例负载与缓存位置选择执行目标，将状态复用机会纳入调度决策。
- **规划与执行分离**：将实例选择与资源预留、状态导入、引擎调用分开；执行前重新校验，失败时按预定范围回退并清理资源，批次调度仍由引擎负责。

架构设计建议

向 SGLang 提出缓存数据面与 Scheduler 分离的 RFC（#37372），讨论元数据所有权、GPU 内存 IPC 与生命周期；另建议将共享缓存传输类型迁到中立模块并兼容导出（#38889）。

技术技能

编程语言：C/C++、Python、Go、Rust、Shell
性能与系统：Linux、perf、Intel Pin、ChampSim、gdb、Paxos
工程工具：Docker、Kubernetes、Prometheus、Grafana、Git、CMake

教育背景

南京大学 2017.09 - 2021.06

环境工程 · 学士 GPA 4.2/5.0